found” errors, which will be confusing for the user. You should also be aware that all these built-in constraints assume invariant culture, which may prove to be problematic if your application uses URLs localized for other languages.

Constraints are best used sparingly, but they can be useful when you have strict requirements on the URLs used by the application, as they can allow you to work around some otherwise-tricky combinations. You can even create custom constraints, as described in the documentation at http://mng.bz/d14Q.

Constraints and overlapping routes

If you have a well-designed set of URLs for your application, you’ll probably find that you don’t need to use route constraints. Route constraints are most useful when you have overlapping route templates.

Suppose that you have an endpoint with the route template `"/{number}/{name}"` and another with the template `"/{product}/{id}"`. When a request with the URL `/shoes/123` arrives, which template is chosen? Both match, so the routing middleware panics and throws an exception—not ideal.

Using constraints can fix this problem. If you update the first template to `"/{number:int}/{name}"`, the integer constraint means that the URL is no longer a match, and the routing middleware can choose correctly. Note, however, that the URL `/123/shoes` still matches both route templates, so you’re not out of the woods.

Generally, you should avoid overlapping route templates like these, as they’re often confusing and more trouble than they’re worth. If your route templates are well defined so that each URL maps to a single template, ASP.NET Core routing will work without any difficulties. Sticking to the built-in conventions as far as possible is the best way to stay on the happy path!

We’re coming to the end of our look at route templates, but before we move on, there’s one more type of parameter to think about: the catch-all